



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

LAB #1 – class 2/5

INTERACTION WITH THE “PHYSICAL” SYSTEM

2025 - 2026

Interaction with physical system

- ☐ Revising the kit/simulator “compact storage”
- ☐ Reading sensors & using the actuators
- ☐ Development of kit interaction routines

Run Project: “labwork1”



Don't FORGET...

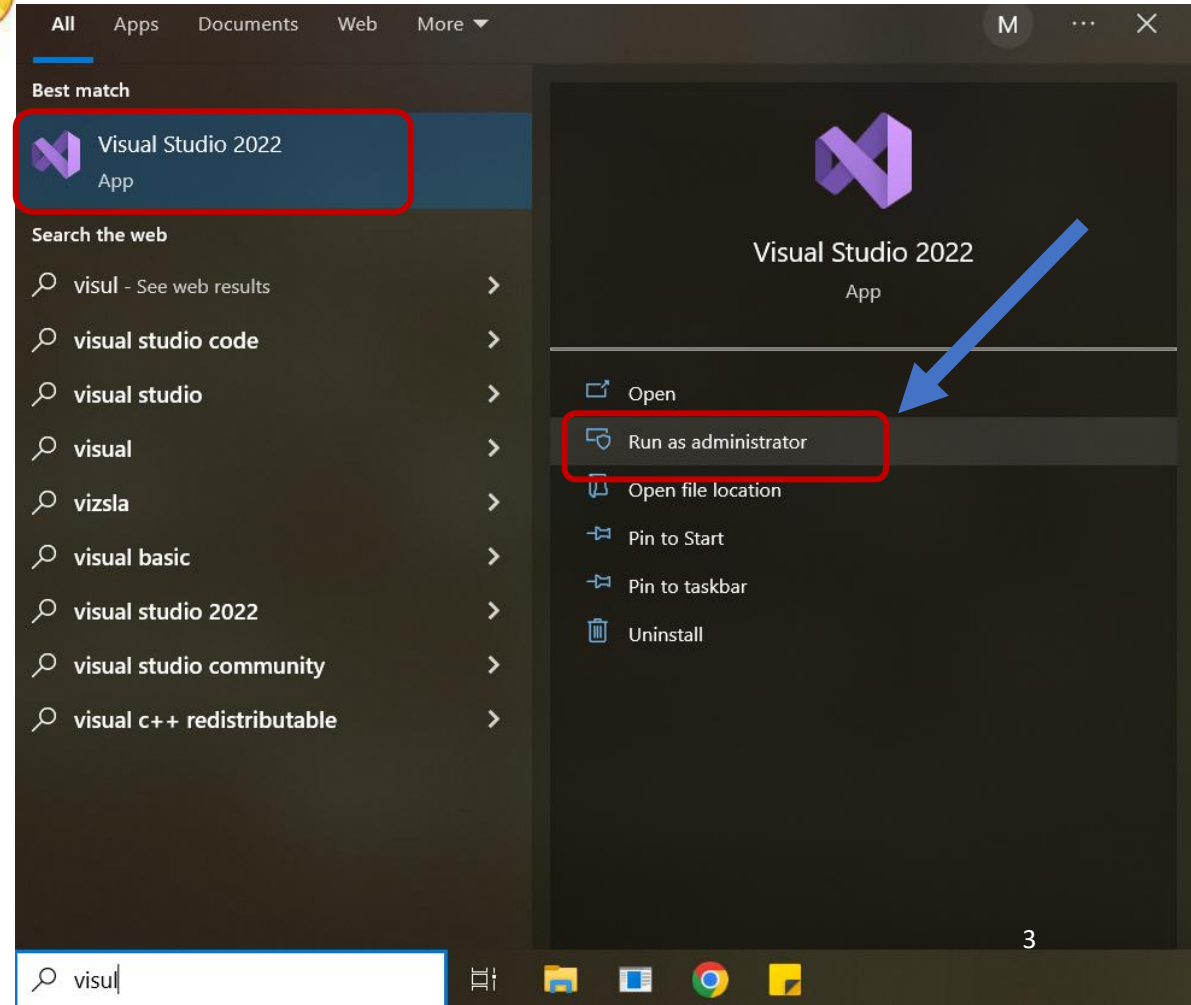


❑ Run VS22

- Find Visual Studio 2022 in your startup menu
- Right click with the mouse
- Select “Run as administrator” (*)

(*) This ensures that our project will obtain more privileges from the Operating System (Windows), ensuring reliable execution in a real time context.

This is important!

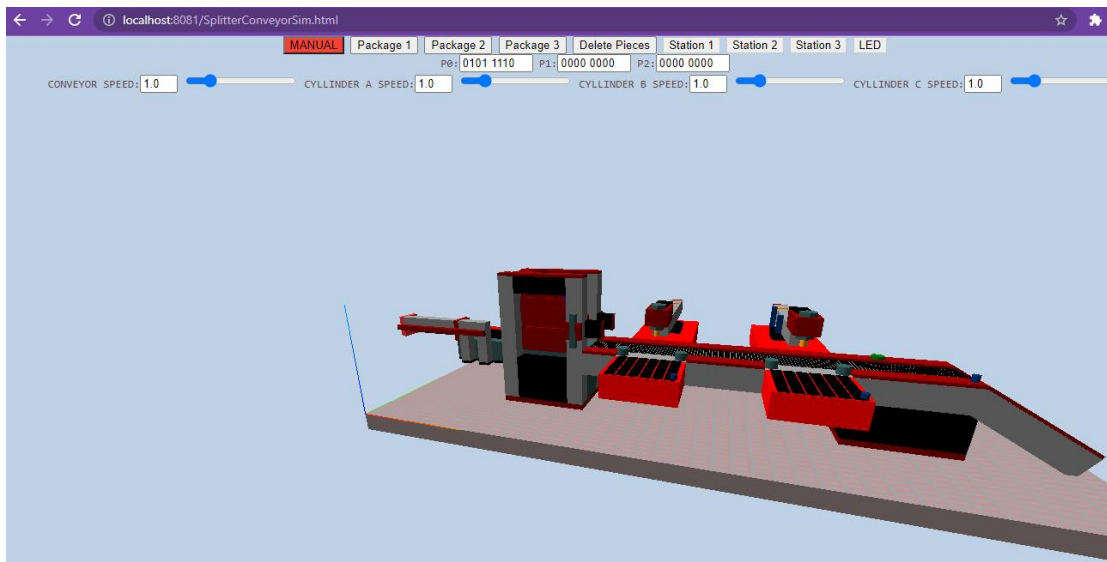


REVIEW Example



- Remember last example from previous class

```
#include<conio.h>
#include<stdlib.h>
#include <windows.h> //for Sleep function
extern "C" {
    #include <interface.h>
}
```



```
int main(int argc, char **argv) {
    createDigitalInput (0);
    createDigitalInput (1);
    createDigitalOutput(2);
    uInt8 p0 = 0;
    uInt8 p2 = 0;
    writeDigitalU8(2, 0x2); // start moving cylinder start front
    while (!_kbhit()) { //do until a key is pressed
        p0 = readDigitalU8(0);
        p2 = readDigitalU8(2);
        if ((p0 & (1 << 5))) { // reached front position?
            p2 &= ~(1 << 1);
            p2 |= (1 << 0);
            writeDigitalU8(2, p2); // move cylinder start back
        }
        if ((p0 & (1 << 6))) { // reached back position?
            p2 &= ~(1 << 0);
            p2 |= (1 << 1);
            writeDigitalU8(2, p2); //move cylinder start front
        }
    }
    writeDigitalU8(2, 0);
    Sleep(5000);
    closeChannels();
}
```

- ❑ It force us to remember each port, each bit and corresponding functionality, e.g., cylinder positions, cylinder movement,... increasing complexity
- ❑ Unsuitable for large systems (e.g., manufacturing systems with thousands sensors /actuators).
- ❑ A good solution is to use a more high-level/"functional" approach, which allows abstraction from the ports, addresses, bits, ... of the many elements in a system

```
int main(int argc, char **argv) {
    createDigitalInput (0);
    createDigitalInput (1);
    createDigitalOutput(2);
    UInt8 p0 = 0;
    UInt8 p2 = 0;
    writeDigitalU8(2, 0x2); // start moving cylinder start front
    while (!_kbhit()) { //do until a key is pressed
        p0 = readDigitalU8(0);
        p2 = readDigitalU8(2);
        if ((p0 & (1 << 5))) { // reached front position?
            p2 &= ~(1 << 1);
            p2 |= (1 << 0);
            writeDigitalU8(2, p2); // move cylinder start back
        }
        if ((p0 & (1 << 6))) { // reached back position?
            p2 &= ~(1 << 0);
            p2 |= (1 << 1);
            writeDigitalU8(2, p2); //move cylinder start front
        }
    }
    writeDigitalU8(2, 0);
    Sleep(5000);
    closeChannels();
}
```

- Example for reading sensors values:

Reading position:


uInt8 p = readDigitalU8(0);

if (getBitValue(p, 5))
printf("\n CylinderStart is at front position);

How to do this?

Reading from port 0,
the value of bit 5

- Example for reading sensors values:

Reading position:

```
uInt8 p = readDigitalU8(0);  
if (getBitValue(p, 5)  
    printf("\n CylinderStart is at front position);
```

Copy this function into your project:

```
int getBitValue(uInt8 value, uInt8 bit_n)  
// given a byte value, returns the value of its bit n  
{  
    return(value & (1 << bit_n));  
}
```

- Example for activating actuators:

Moving cylinder start front:

```
uInt8 p = readDigitalU8(2); //read port 2
setBitValue(&p, 0, 0);      //set bit 0 to low level
setBitValue(&p, 1, 1);      //set bit 1 to high level
writeDigitalU8(2, p);       //update port 2
```

Copy this function into your code:

```
void setBitValue(uInt8 *variable, int n_bit, int new_value_bit)
// given a byte value, set the n bit to value
{
    uInt8 mask_on = (uInt8)(1 << n_bit);
    uInt8 mask_off = ~mask_on;
    if (new_value_bit) *variable |= mask_on;
    else               *variable &= mask_off;
}
```

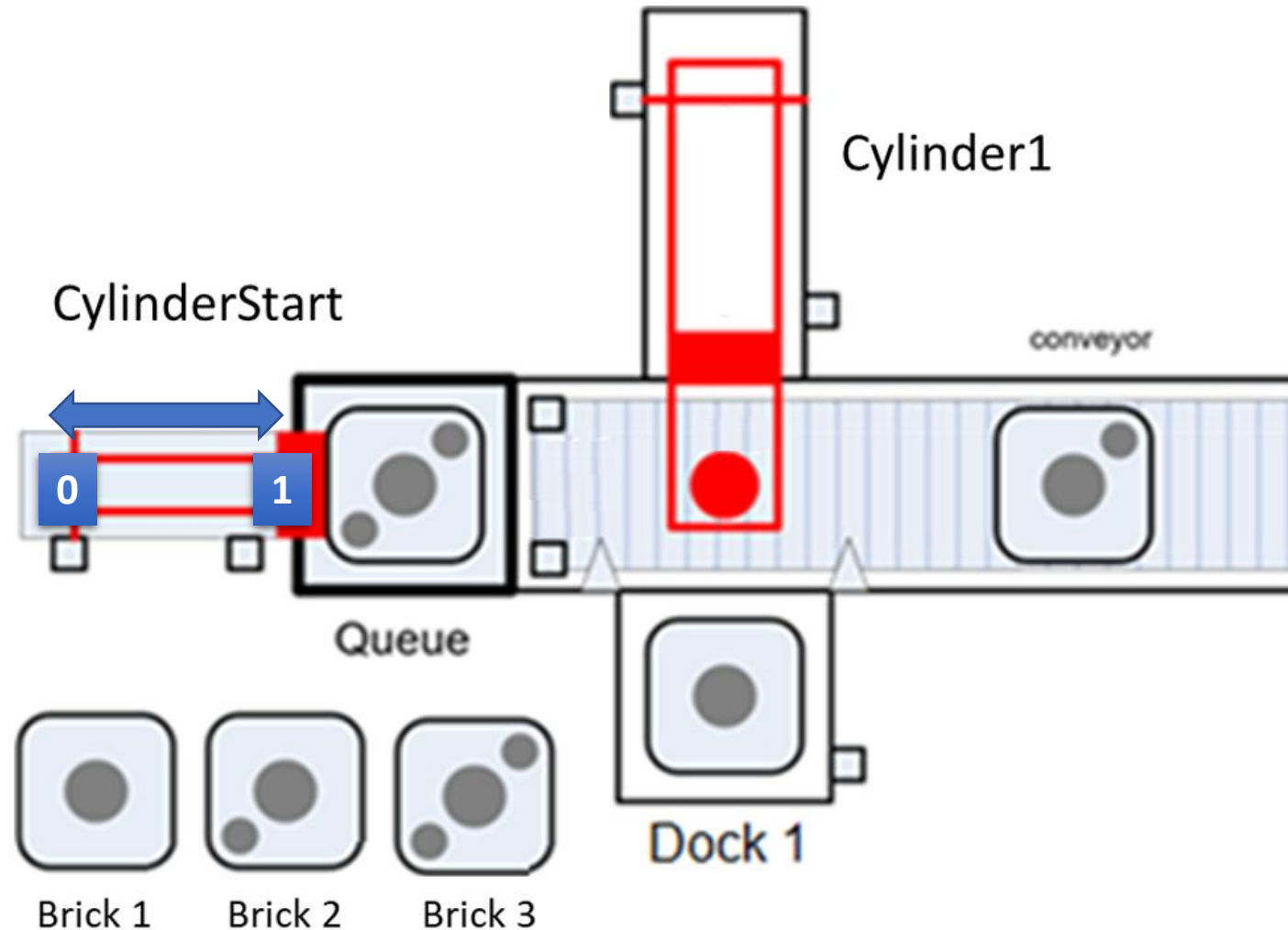

SOME ABSTRACT FUNCTIONALITIES (CYLINDER START)

Interface.c

```
void createDigitalInput (uInt32 c)  
void createDigitalOutput (uInt32 c)  
writeDigitalU8(uInt32 port, uInt8 data)
```

```
int getBitValue(value, n_bit)  
void setBitValue(variable, bit, value)
```

```
void moveCylinderStartFront();  
void moveCylinderStartBack();  
void stopCylinderStart();  
int getCylinderStartPos();  
void gotoCylinderStart(int pos);
```



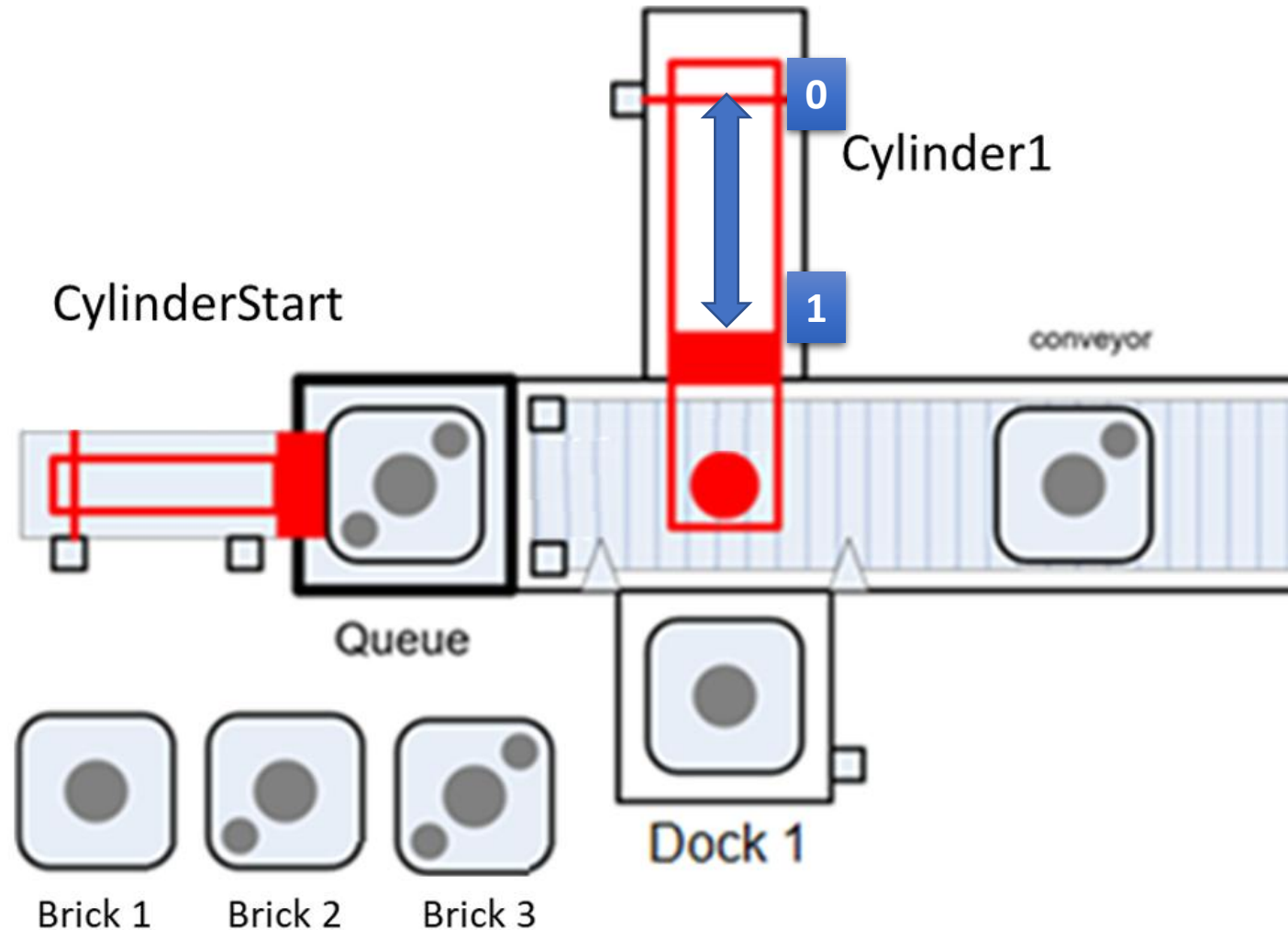
SOME ABSTRACT FUNCTIONALITIES (CYLINDER 1)

Interface.c

```
void createDigitalInput (uInt32 c)
void createDigitalOutput (uInt32 c)
writeDigitalU8(uInt32 port, uInt8 data)
```

```
int getBitValue(value, n_bit)
void setBitValue(variable, bit, value)
```

```
void moveCylinder1Front();
void moveCylinder1Back();
void stopCylinder1();
int getCylinder1Pos();
void gotoCylinder1(int pos);
```



SOME ABSTRACT FUNCTIONALITIES (CYLINDER 2)

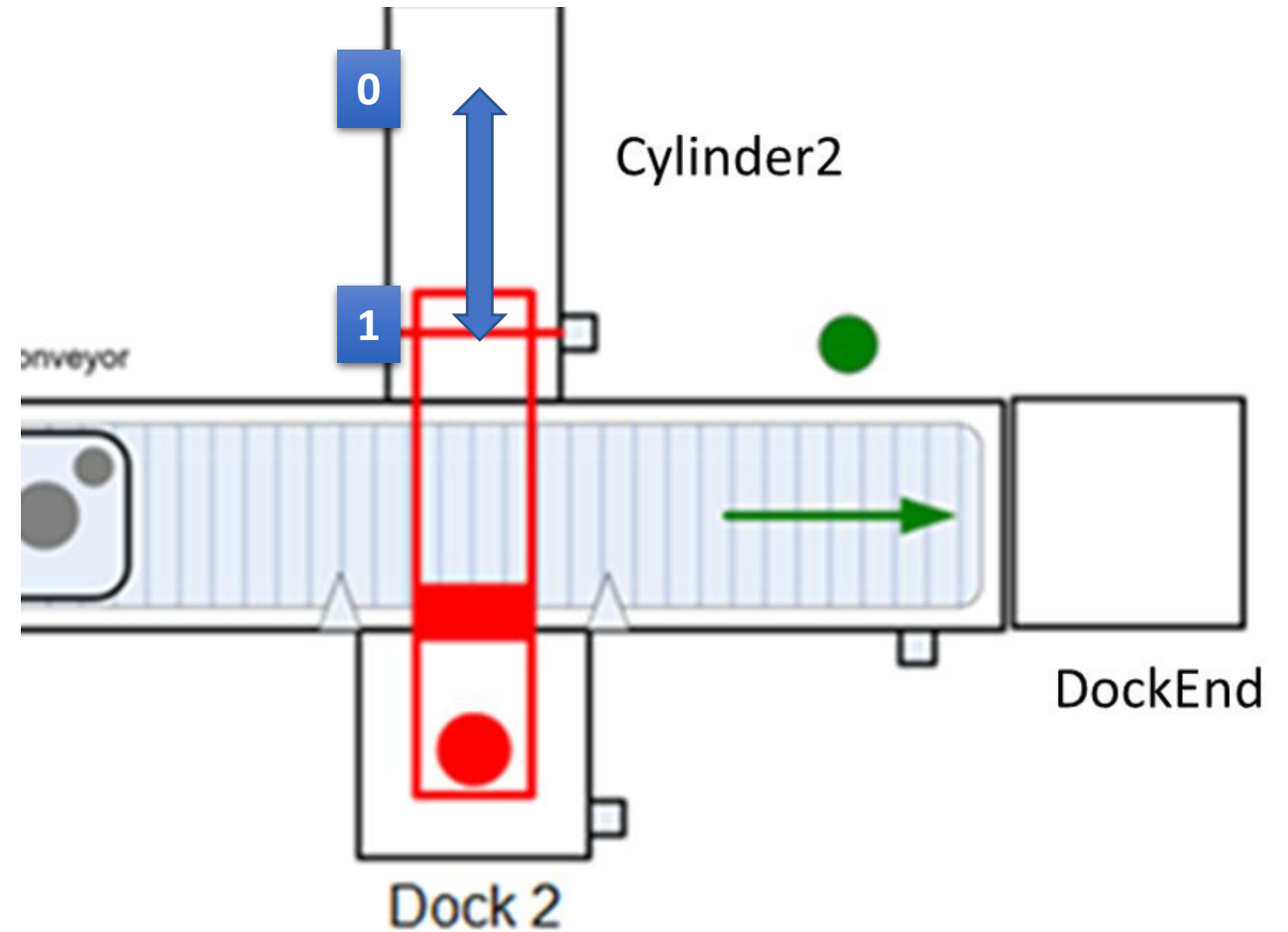


Interface.c

```
void createDigitalInput (uInt32 c)
void createDigitalOutput (uInt32 c)
writeDigitalU8(uInt32 port, uInt8 data)
```

```
int getBitValue(value, n_bit)
void setBitValue(variable, bit, value)
```

```
void moveCylinder2Front();
void moveCylinder2Back();
void stopCylinder2();
int getCylinder2Pos();
void gotoCylinder2(int pos);
```



SOME ABSTRACT FUNCTIONALITIES (OTHERS)

Interface.c

```
void createDigitalInput (uInt32 c)
void createDigitalOutput (uInt32 c)
writeDigitalU8(uInt32 port, uInt8 data)
```

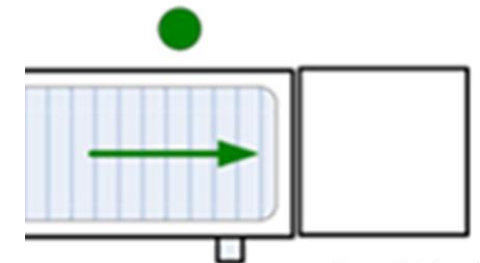
```
int getBitValue(value, n_bit)
void setBitValue(variable, bit, value)
```

```
void moveCylinderStartFront();
void moveCylinderStartBack();
void stopCylinderStart();
int getCylindersPos();
void gotoCylinderStart(int pos);
```

```
void moveCylinder1Front();
void moveCylinder1Back();
void stopCylinder1();
int getCylinder1Pos();
void gotoCylinder1(int pos);
```

```
void moveCylinder2Front();
void moveCylinder2Back();
void stopCylinder2();
int getCylinder2Pos();
void gotoCylinder2(int pos);
```

```
void moveConveyor();
void stopConveyor();
void setLed(int on);
```

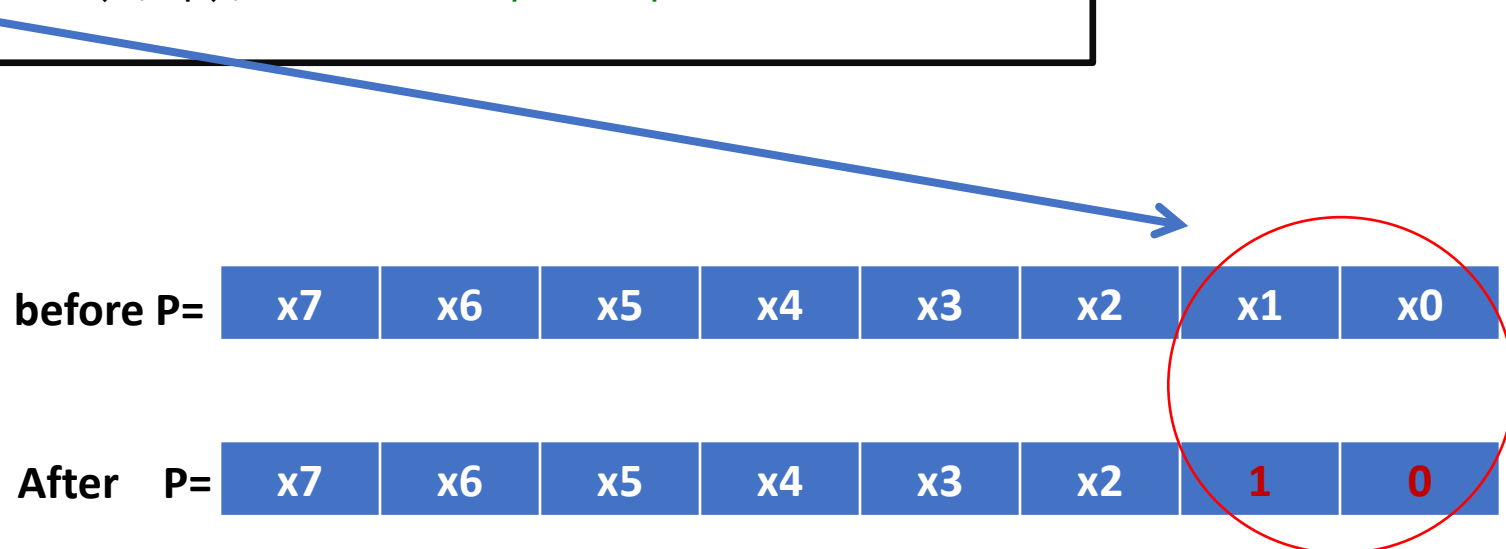


DockEnd

❑ Function `moveCylinderStartFront()` [similar for Cylinder 1 and 2]

```
void moveCylinderStartFront()
{
    uInt8 p = readDigitalU8(2); // read port 2
    setBitValue(&p, 0, 0);      // set bit 0 to low level
    setBitValue(&p, 1, 1);      // set bit 1 to high level
    writeDigitalU8(2, p);       // update port 2
}
```

before P=



x7	x6	x5	x4	x3	x2	x1	x0
----	----	----	----	----	----	----	----

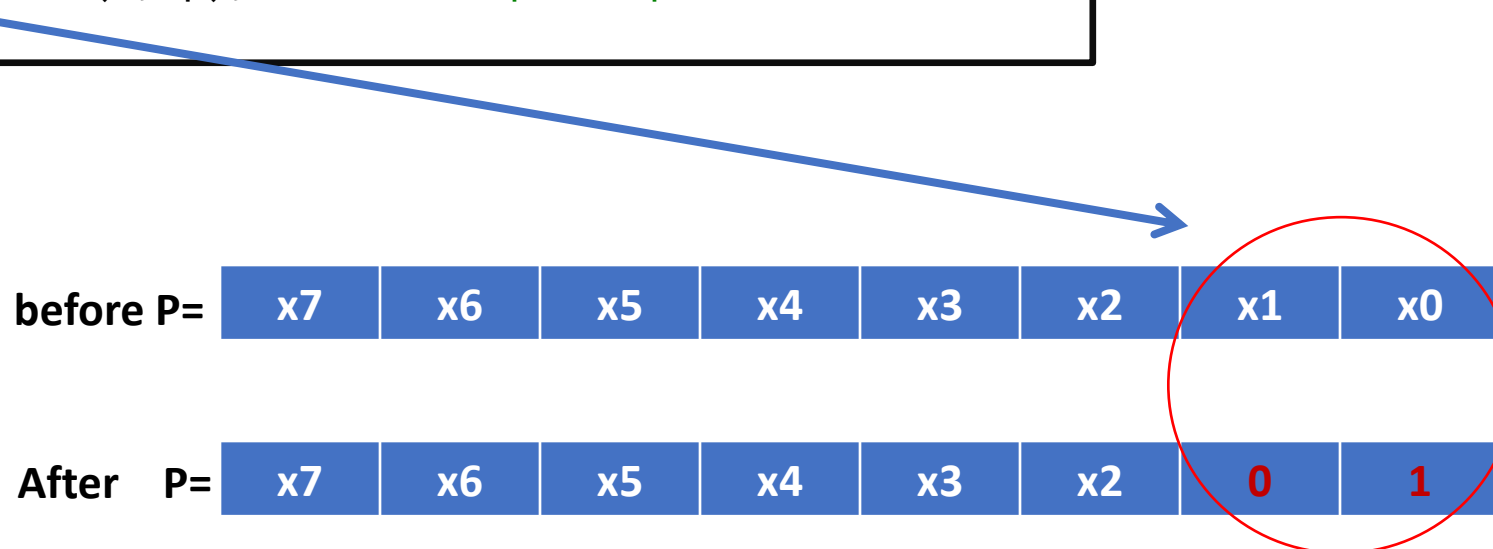
After P=

x7	x6	x5	x4	x3	x2	1	0
----	----	----	----	----	----	---	---

❑ Function `moveCylinderStartBack()` [similar for Cylinder 1 and 2]

```
void moveCylinderStartBack()
{
    uInt8 p = readDigitalU8(2); // read port 2
    setBitValue(&p, 0, 1);      // set bit 0 to high level
    setBitValue(&p, 1, 0);      // set bit 1 to low level
    writeDigitalU8(2, p);       // update port 2
}
```

before P=



x7	x6	x5	x4	x3	x2	x1	x0
----	----	----	----	----	----	----	----

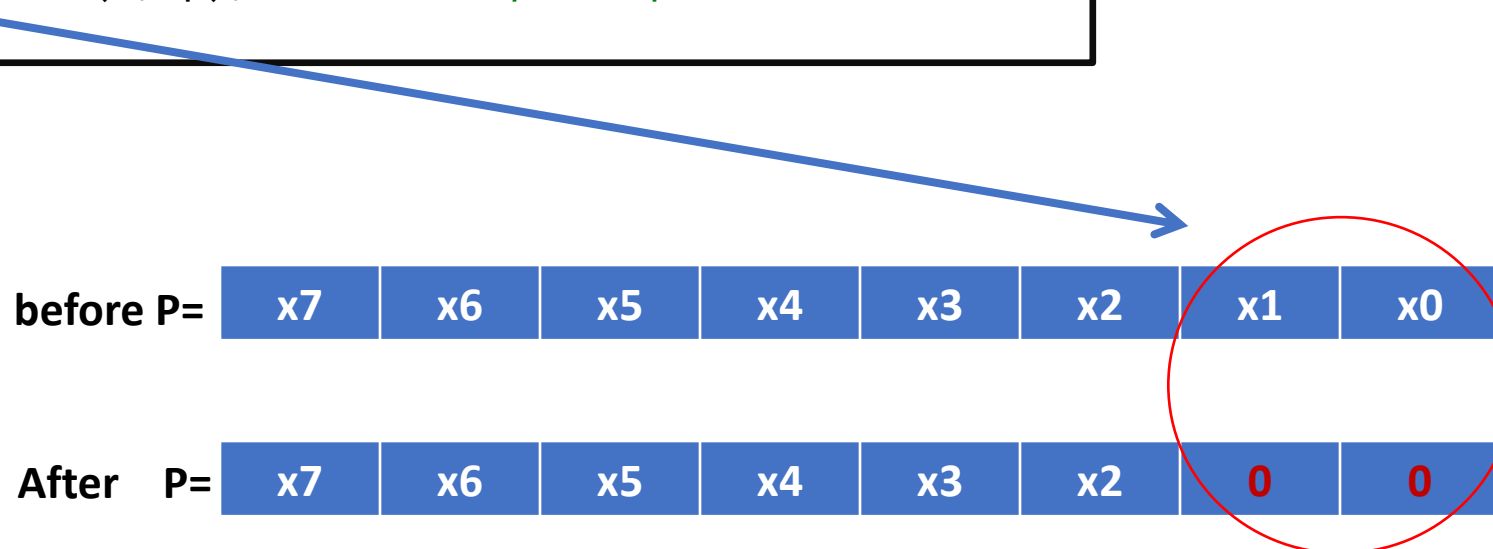
After P=

x7	x6	x5	x4	x3	x2	0	1
----	----	----	----	----	----	---	---

- ❑ Function [stopCylinderStart\(\)](#) [similar for Cylinder 1 and 2]

```
void stopCylinderStart()
{
    uInt8 p = readDigitalU8(2); // read port 2
    setBitValue(&p, 0, 0);      // set bit 0 to low level
    setBitValue(&p, 1, 0);      // set bit 1 to low level
    writeDigitalU8(2, p);       // update port 2
}
```

before P=



x7	x6	x5	x4	x3	x2	x1	x0
----	----	----	----	----	----	----	----

After P=

x7	x6	x5	x4	x3	x2	0	0
----	----	----	----	----	----	---	---

☐ Read CylinderStart, Cylinder1 and Cylinder2 positions

```
int getCylinderStartPos()
{
    uInt8 p0 = readDigitalU8(0);
    if( getBitValue(p0,6) )
        return 0;
    if( getBitValue(p0,5) )
        return 1;
    return(-1);
}
```

```
int getCylinder1Pos()
{
    //(...) Students' job
}
```

```
int getCylinder2Pos()
{
    uInt8 p0 = readDigitalU8(0);
    if( !getBitValue(p0,2) )
        return 0;
    if( !getBitValue(p0,1) )
        return 1;
    return(-1);
}
```


LABWORK1 PROJECT: Work Assignment

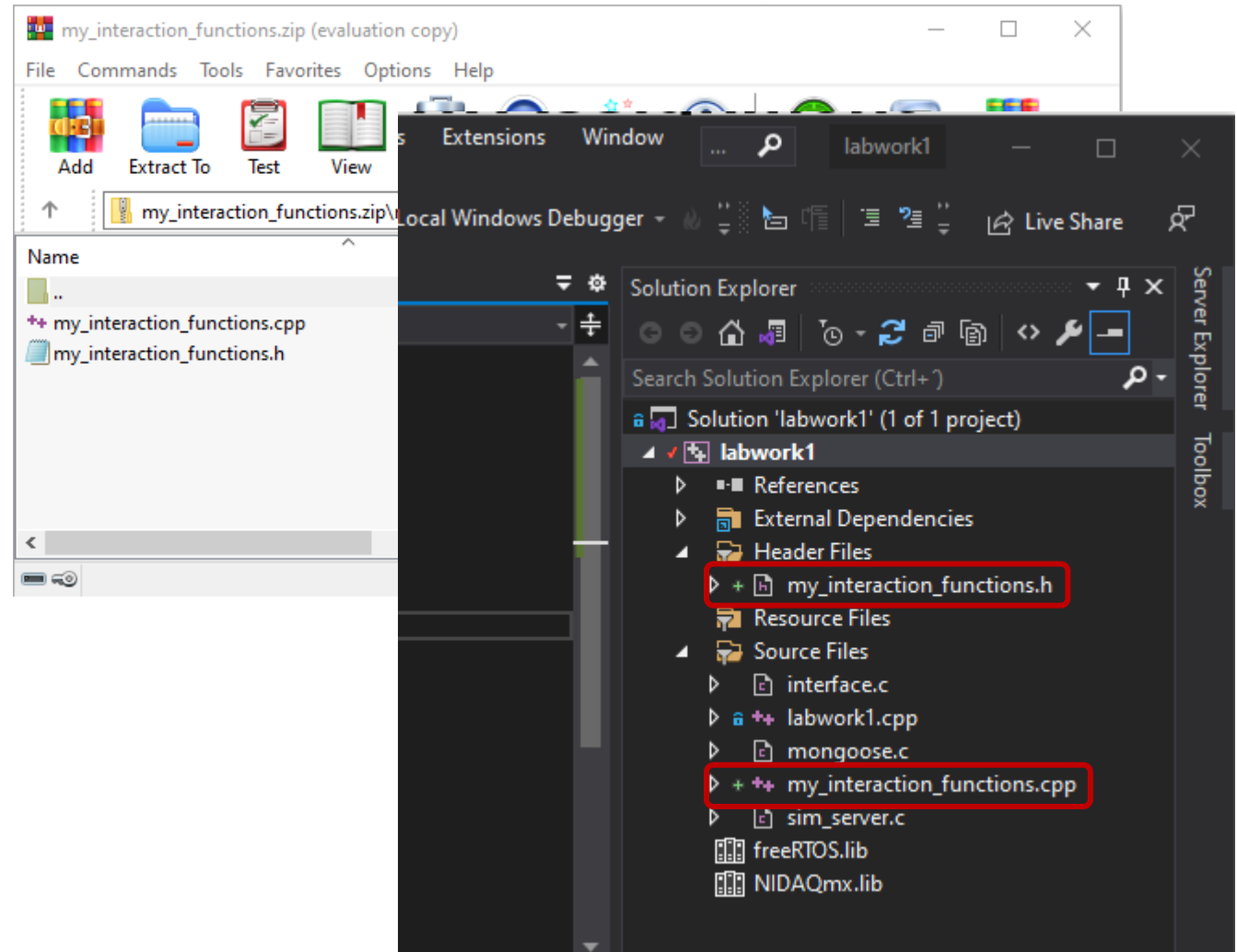


- ❑ Considering the given implementation examples, **develop all the functions specified in the previous slides.**

Suggestion:



- ❑ Extract the **my_interaction_functions.zip** file from CLIP
- ❑ Add the **.cpp** and the **.h** files to the labwork1 project
- ❑ Complete the implementation of the functions



LABWORK1 PROJECT: Work Assignment



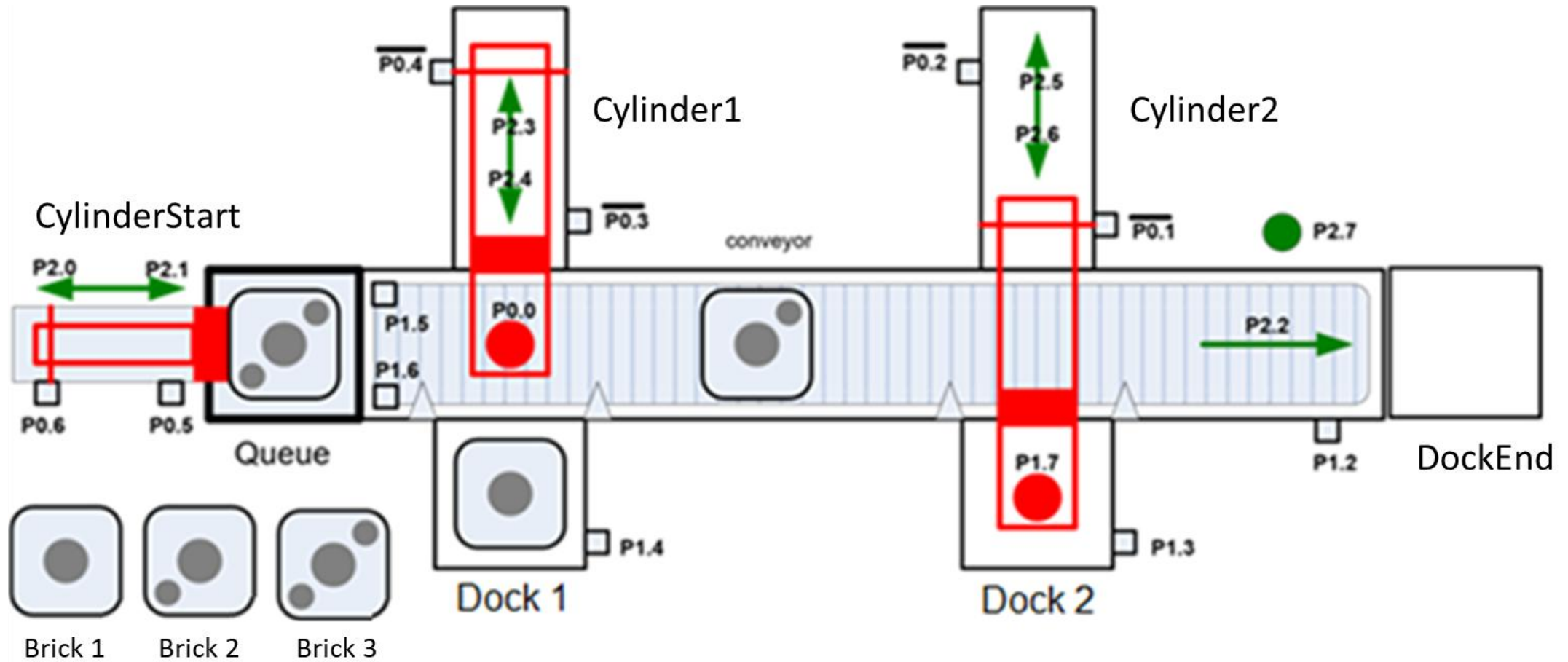
■ In labwork1.cpp

- Create ports
- Launch and manually calibrate the kit simulator
- Test each function:
gotoCylinderStart(int pos), gotoCylinder1, ... ,

```
int main()
{
    // INPUT PORTS
    createDigitalInput(0);
    createDigitalInput(1);
    // OUTPUT PORTS
    createDigitalOutput(2);

    printf("\ncallibrate kit manually and press enter...");

    int tecla = 0;
    while (tecla != 27) {
        tecla = _getch();
        if (tecla == 'q')
            moveCylinderStartFront();
        if (tecla == 'z')
            printf("vai mover cylinder Start back");
            // moveCylinderStartBack();
        if (tecla == 'a')
            stopCylinderStart();
        if (tecla == 'w')
            printf(" vai mover Cylinder1 front");
            // (...)
        if (tecla == 'x')
            printf(" vai mover Cylinder1 back");
            // (...)
        if // (...)
        // to complete...
    }
    closeChannels();
    return 0;
}
```



AWESOME! YOU'RE ALL SET TO START PLAYING WITH YOUR LAB 1 SYSTEM IN REAL-TIME SYSTEMS!



RELAX, EXPLORE, HAVE FUN, AND ENJOY THE RIDE!



KEEP UP THE
GOOD
WORK!